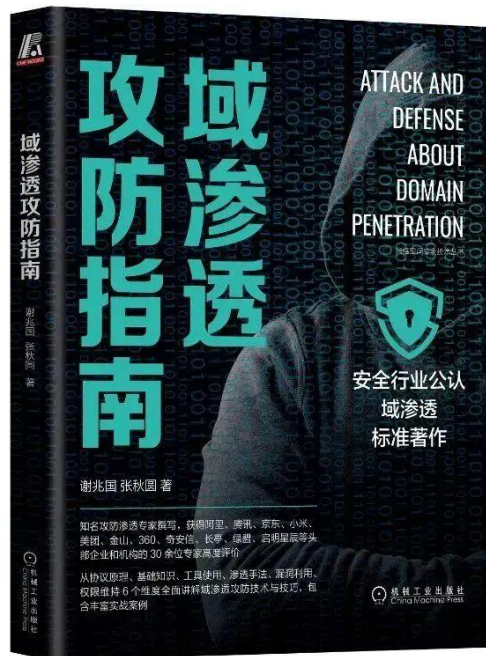


Kerberos 协议详解

本文部分节选自《域渗透攻防指南》，购买请长按如下图片扫码



小谢1997521 为你推荐

【限量签名版】《域渗透攻防指南》

¥99 ¥129



本商品由 放心购 提供保障

谢公子学安全

Kerberos 协议是由麻省理工学院提出的一种网络身份验证协议，提供了一种在开放的非安全网络中认证识别用户身份信息的方法。它旨在通过使用密钥加密技术为客户端/服务端应用程序提供强身份验证。Kerberos 是西方神话中守卫地狱之门的三头犬的名字。之所以使用这个名字是因为 Kerberos 需要三方的共同参与才能完成一次认证流程。目前主流使用的 Kerberos 版本为 2005 年 RFC4120(<https://www.rfc-editor.org/rfc/rfc4120>)

c4120.html)标准定义的 KerberosV5 版本，Windows、Linux 和 Mac OS 均支持 Kerberos 协议。

Kerberos 基础

在 Kerberos 协议中，主要有以下三个角色：

- 访问服务的客户端：Kerberos 客户端是代表需要访问资源的用户进行操作的应用程序，例如打开文件、查询数据库或打印文档。每个 Kerberos 客户端在访问资源之前都会请求身份验证。
- 提供服务的服务端：域内提供服务的服务端，服务端都有一个独一的 SPN。
- 提供认证服务的 KDC(Key Distribution Center, 密钥分发中心)：KDC 密钥发行中心是一种网络服务，它向活动目录域内的用户和计算机提供会话票据和临时会话密钥，其服务帐户为 krbtgt。KDC 作为活动目录域服务 ADDS 的一部分运行在每个域控制器上。

这里说一下 krbtgt 帐户，该用户是在创建活动目录时系统自动创建的一个账号，其作用是 KDC 密钥发行中心的服务账号，其密码是系统随机生成的，无法正常登陆主机。在后面的域学习中，会经常和 krbtgt 帐户打交道，关于该帐户的其他信息会在后面的文章中详细讲解。

如图所示，可以看到 krbtgt 帐户的信息。

```
C:\>net user krbtgt
User name                krbtgt
Full Name                密钥发行中心服务帐户
Comment                  密钥发行中心服务帐户
User's comment
Country/region code      000 (System Default)
Account active           No
Account expires          Never

Password last set        2021/6/30 18:33:10
Password expires         2021/8/11 18:33:10
Password changeable      2021/7/1 18:33:10
Password required        Yes
User may change password Yes

Workstations allowed     All
Logon script
User profile
Home directory
Last logon               Never

Logon hours allowed      All

Local Group Memberships  *Denied RODC Password
Global Group memberships *Domain Users
The command completed successfully.
```

Kerberos 是一种基于票据 Ticket 的认证方式。客户端想要访问服务端的某个服务，首先需要购买服务端认可的 **ST 服务票据(Service Ticket)**。也就是说，客户端在访问服务之前需要先买好票，等待服务验票之后才能访问。但是这张票并不能直接购买，需要一张 **TGT 认购权证** (Ticket Granting Ticket)。也就是说，客户端在买票之前必须先获得一张 TGT 认购权证。**TGT 认购权证** 和 **ST 服务票据** 都是由 KDC(密钥分发中心)发放；因为 KDC 是运行在域控制器上，所以说 TGT 认购权证和 ST 服务票据均是由域控发放。

Kerberos 使用 TCP/UDP 88 端口进行认证，使用 TCP/UDP 464 端口进行密码重设。

如图所示，可以看到域控制器 AD01 上开放的 88 和 464 端口。

```
PORT      STATE SERVICE      VERSION
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2021-12-26 09:47:09Z)
389/tcp   open  ldap         Microsoft Windows Active Directory LDAP (Domain: xie.com, Site: Default-First-Site-Name)
464/tcp   open  kpasswd5?    Service Info: Host: AD01; OS: Windows; CPE: cpe:/o:microsoft:windows
```

Kerberos 中一些名词的简称及含义如表 1-1 所示：

简称 全拼

DC	Domain Controller，域控
krbtgt	KDC 密钥发行中心服务账户
KDC	Key Distribution Center：密钥分发中心，由域控担任
AD	Active Directory：活动目录，里面包含域内用户数据库
AS	Authentication Service：认证服务
TGT	Ticket Granting Ticket：TGT 认购权证，由 KDC 的 AS 认证服务发放
TGS	Ticket Granting Service：票据授予服务
ST	Service Ticket：ST 服务票据，由 KDC 的 TGS 票据授予服务发放

表 1-1 名词简称及含义

Kerberos 协议有两个基础认证模块：AS_REQ & AS_REP 和 TGS_REQ & TGS_REP，以及微软扩展的两个认证模块 S4U 和 PAC。S4U 是微软为了实现委派而扩展的模块，分为 S4U2Self 和 S4U2Proxy。在 Kerberos 最初设计的流程里只说明了如何证明客户端的真实身份，但是并没有说明客户端是否有权限访问该服务，因为在域中不同权限的用户能够访问的资源是不同的。因此微软为了解决权限这个问题，引入了 PAC (Privilege Attribute Certificate，特权属性证书) 的概念。

在分析 AS_REQ & AS_REP 和 TGS_REQ & TGS_REP 之前，我们先来看看什么是 PAC。

PAC 特权属性证书

PAC (Privilege Attribute Certificate, 特权属性证书), 其中所包含的是各种授权信息、附加凭据信息、配置文件和策略信息等。例如用户所属的用户组, 用户所具有的权限等。在最初的 RFC1510 中规定的标准 Kerberos 认证过程中并没有 PAC, 微软在自己的产品中所实现的 Kerberos 流程加入了 PAC 的概念, 因为在域中不同权限的用户能够访问的资源是不同的, 因此微软设计 PAC 用来辨别用户身份和权限。

在一个正常的 Kerberos 认证流程中, KDC 返回的 TGT 认购权证和 ST 服务票据中都是带有 PAC 的。这样做的好处就是在以后对资源的访问中, 服务端再接收到客户请求的时候不再需要借助 KDC 的帮助提供完整的授权信息来完成对用户权限的判断, 而只需要根据请求中所包含的 PAC 信息直接与本地资源的 ACL 相比较做出裁决。

1. PAC 结构

PAC 的顶部结构如下:

```
typedef unsigned long ULONG;  
typedef unsigned short USHORT;  
typedef unsigned long64 ULONG64;  
typedef unsigned char UCHAR;
```

```
typedef struct _PACTYPE {  
    ULONG cBuffers;  
    ULONG Version;  
    PAC_INFO_BUFFER Buffers[1];  
} PACTYPE;
```


- Sserver Cechksum (6)
- Privsvr Cechksum (7)
- **cbBufferSize**: 缓冲大小
- **Offset**: 缓冲偏移量

如图所示，是 WireShark 抓包的 PAC_INFO_BUFFER 结构部分。

```

v Type: Logon Info (1)
  Size: 432
  Offset: 88
  > PAC_LOGON_INFO: 01100800ccccccca001000000000000000020091d2b94b97f5d701ffffffffffff7f...
v Type: Client Info Type (10)
  Size: 18
  Offset: 520
  > PAC_CLIENT_INFO_TYPE: 00409e7997f5d70108006800610063006b00
v Type: UPN DNS Info (12)
  Size: 56
  Offset: 544
  > UPN_DNS_INFO: 180010000e00280001000000000000006800610063006b0040007800690065002e006300...
v Type: Server Checksum (6)
  Size: 16
  Offset: 600
  > PAC_SERVER_CHECKSUM: 1000000066c45acb1e6c3977cefabbc1
v Type: Privsvr Checksum (7)
  Size: 20
  Offset: 616
  > PAC_PRIVSVR_CHECKSUM: 76ffffff2ca125e6b1d17dfe8d6ce4f0d967cc25

```

2. PAC 凭证信息

LOGON INFO 类型的 PAC_LOGON_INFO 包含 Kerberos 票据客户端的凭证信息。数据本身包含在一个 KERB_VALIDATION_INFO 结构中，该结构是由 NDR 编码的。NDR 编码的输出被放置在 LOGON INFO 类型的 PAC_INFO_BUFFER 结构中。如下：

```

typedef struct _KERB_VALIDATION_INFO {
    FILETIME Reserved0;
    FILETIME Reserved1;
    FILETIME KickOffTime;
    FILETIME Reserved2;

```

```

FILETIME Reserved3;
FILETIME Reserved4;
UNICODE_STRING Reserved5;
UNICODE_STRING Reserved6;
UNICODE_STRING Reserved7;
UNICODE_STRING Reserved8;
UNICODE_STRING Reserved9;
UNICODE_STRING Reserved10;
USHORT Reserved11;
USHORT Reserved12;
ULONG UserId;
ULONG PrimaryGroupId;
ULONG GroupCount;
[size_is(GroupCount)] PGROUP_MEMBERSHIP GroupIds;
ULONG UserFlags;
ULONG Reserved13[4];
UNICODE_STRING Reserved14;
UNICODE_STRING Reserved15;
PSID LogonDomainId;
ULONG Reserved16[2];
ULONG Reserved17;
ULONG Reserved18[7];
ULONG SidCount;
[size_is(SidCount)] PKERB_SID_AND_ATTRIBUTES ExtraSids;
PSID ResourceGroupDomainSid;
ULONG ResourceGroupCount;
[size_is(ResourceGroupCount)] PGROUP_MEMBERSHIP ResourceGroupIds;
} KERB_VALIDATION_INFO;

```

主要还是关注以下几个字段：

- Acct Name：该字段对应的值是用户 sAMAccountName 属性的值

- Full Name: 该字段对应的值是用户 displayName 属性的值
- User RID: 该字段对应的值是用户的 RID, 也就是用户 SID 的最后部分
- Group RID: 对于该字段, 域用户的 Group RID 恒为 513(也就是 Domain Users 的 RID), 机器用户的 Group RID 恒为 515(也就是 Domain Computers 的 RID), 域控的 Group RID 恒为 516(也就是 Domain Controllers 的 RID)
- Num RIDs: 用户所属组的个数
- GroupIDs: 用户所属的所有组的 RID

如图所示, 是是 WireShark 抓包的 PAC_LOGON_INFO 部分。



```

✓ PAC_LOGON_INFO: 01100800ccccccca001000000000000000020091d2b94b97f5d701ffffffffffff7f...
  > MES header
  ✓ PAC_LOGON_INFO:
    Referent ID: 0x00020000
    Logon Time: Dec 20, 2021 19:46:59.005505700 CST
    Logoff Time: Infinity (absolute time)
    Kickoff Time: Infinity (absolute time)
    PWD Last Set: Jul 1, 2021 21:25:09.806508900 CST
    PWD Can Change: Jul 2, 2021 21:25:09.806508900 CST
    PWD Must Change: Infinity (absolute time)
  > Acct Name: hack
  > Full Name: hack
  > Logon Script
  > Profile Path
  > Home Dir
  > Dir Drive
  Logon Count: 28
  Bad PW Count: 0
  User RID: 1154
  Group RID: 513
  Num RIDs: 1
  ✓ GroupIDs
    Referent ID: 0x0002001c
    Max Count: 1
    > GROUP_MEMBERSHIP:
  > User Flags: 0x00000020
  User Session Key: 00000000000000000000000000000000
  > Server: AD01
  > Domain: XIE
  > SID pointer:
    Dummy1 Long: 0x00000000
    Dummy2 Long: 0x00000000
  > User Account Control: 0x00000010
    Dummy4 Long: 0x00000000
    Dummy5 Long: 0x00000000
    Dummy6 Long: 0x00000000
    Dummy7 Long: 0x00000000
    Dummy8 Long: 0x00000000
    Dummy9 Long: 0x00000000
    Dummy10 Long: 0x00000000
    Num Extra SID: 1
  ✓ SID_AND_ATTRIBUTES_ARRAY:
    Referent ID: 0x0002002c
    > SID_AND_ATTRIBUTES array:
  > ResourceGroupIDs

```

3. PAC 签名

PAC 中包含两个数字签名：PAC_SERVER_CHECKSUM 和 PAC_PRIVSVR_CHECKSUM。PAC_SERVER_CHECKSUM 是使用服务密钥进行签名，而 PAC_PRIVSVR_CHECKSUM 是使用 KDC 密钥进行签名。签名有两个原因。首先，存在带有服务密钥的签名，以验证此 PAC 由服务进行了签名。其次，带有 KDC 密钥的签名是为了防止不受信任的服务用无效的 PAC 为自己伪造

票据。这两个签名分别以 PAC_SERVER_CHECKSUM 和 PAC_PRIVSVR_CHECKSUM 类型的 PAC_INFO_BUFFER 发送。在 PAC 数据用于访问控制之前，必须检查 PAC_SERVER_CHECKSUM 签名。这将验证客户端是否知道服务的密钥。而 PAC_PRIVSVR_CHECKSUM 签名的验证是可选的，默认不开启。它用于验证 PAC 是否由 KDC 签发，而不是由 KDC 以外的具有访问服务密钥的人放入票据中。

签名包含在以下结构中：

```
typedef struct _PAC_SIGNATURE_DATA {  
    ULONG SignatureType;  
    UCHAR Signature[1];  
} PAC_SIGNATURE_DATA, *PPAC_SIGNATURE_DATA;
```

这些字段定义如下：

- SignatureType：此字段包含用于创建签名的校验和的类型，校验和必须是一个键控的校验和。
- Signature：此字段由一个包含校验和数据的字节数组组成。字节的长度可以由包装 PAC_INFO_BUFFER 结构来决定。

如图所示，是 PAC 中的签名部分，可以看到 PAC_SERVER_CHECKSUM 和 PAC_PRIVSVR_CHECKSUM。

- ✓ Type: Server Checksum (6)
 - Size: 16
 - Offset: 600
 - ✓ PAC_SERVER_CHECKSUM: 1000000066c45acb1e6c3977cefabb1
 - Type: 16
 - Signature: 66c45acb1e6c3977cefabb1
- ✓ Type: Privsvr Checksum (7)
 - Size: 20
 - Offset: 616
 - ✓ PAC_PRIVSVR_CHECKSUM: 76ffffff2ca125e6b1d17dfe8d6ce4f0d967cc25
 - Type: -138
 - Signature: 2ca125e6b1d17dfe8d6ce4f0d967cc25

4. KDC 验证 PAC

我们注意到，PAC 中是有两个签名的：PAC_SERVER_CHECKSUM 和 PAC_PRIVSVR_CHECKSUM。一个是使用服务密钥(PAC_SERVER_CHECKSUM)进行签名，另一个使用 KDC 密钥(PAC_PRIVSVR_CHECKSUM)进行签名。当服务端收到客户端发来的 AP-REQ 消息时，只能校验 PAC_SERVER_CHECKSUM 签名，而并不能校验 PAC_PRIVSVR_CHECKSUM 签名。因此，正常来说如果需要校验 PAC_PRIVSVR_CHECKSUM 签名的话，服务端还需要将客户端发来的 ST 服务票据中的 PAC 签名发给 KDC 进行校验。但是，由于大部分服务默认并没有 KDC 验证 PAC 这一步(需要将目标服务主机配置为验证 KDC PAC 签名，默认未开启)，因此服务端就无需将 ST 服务票据中的 PAC 签名发给 KDC 校验了。这也是白银票据攻击能成功的前提，因为如果配置了需要验证 PAC_PRIVSVR_CHECKSUM 签名的话，服务端会将这个 PAC 的数字签名以 KRB_VERIFY_PAC 的消息通过 RPC 协议发送给 KDC，KDC 再将验证这个 PAC 的数字签名的结果以 RPC 返回码的形式发送给服务端，服务端就可以根据这个返回结果判断 PAC 的真实性和有效性了。因此如果目标服务主机配置了要校验 PAC_PRIVSVR_CHECKSUM 签名的话，就算攻击者拥有服务密钥，可以制作 ST 服务票据，也不能伪造 KDC 的 PAC_PRIVSVR_CHECKSUM 签名，自然就无法通过 KDC 的签名校验了。

那么如何配置服务主机开启 KDC 签名校验呢，根据微软官方文档的描述，若要开启 KDC 校验 PAC，需要有以下条件：

- 应用程序具有 SeTcbPrivilege 权限。SeTcbPrivilege 权限允许为用户帐户分配“作为操作系统的一部分”。本地系统、网络服务和本地服务帐户都是由 windows 定义的服务用户帐户。每个帐户都有一组特定的特权。
- 应用程序是一个服务，验证 KDC PAC 签名的注册表项被设置为 1，默认为 0。修改方法如下：

1.

1. 启动注册表编辑器 regedit.exe
2. 找到以下子键：
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters
3. 添加一个 ValidateKdcPacSignature 的键值(DWORD 类型)。该值为 0 时，不会进行 KDC PAC 校验。该值为 1 时，会进行 KDC PAC 校验。因此可以将该值设置为 1 启用 KDC PAC 校验。

对于验证 KDC PAC 签名这个注册表键值，有以下几点注意事项：

- 如果服务端并非一个服务程序，而是一个普通应用程序，它将不受以上注册表的影响，而总是进行 KDC PAC 校验。
- 如果服务端并非一个程序，而是一个驱动，其认证过程在系统内核内完成，它将不受以上注册表的影响，而永不进行 PAC 校验。
- 使用以上注册表项，需要在 Windows Server 2003 SP2 或更新的操作系统。
- 在运行 Windows Server 2008 或更新操作系统的服务器上，该注册表项的值缺省为 0(默认没有该 ValidateKdcPacSignature 键值)，也就是不进行 KDC PAC 校验。

注：需要说明的是，注册在本地系统帐户下的服务无论如何配置，都不会触发 KDC 验证 PAC 签名。也就是说譬如 SMB、CIFS、HOST 等服务无论如何都不会触发 KDC 验证 PAC 签名。

那么为什么默认情况下，KDC 都不会验证 PAC 的签名呢？

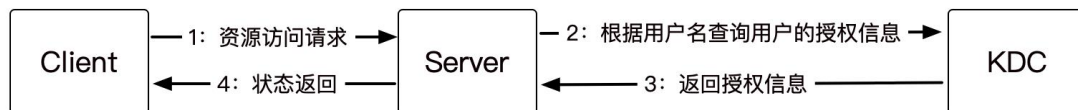
执行 KDC 验证 PAC 的话，意味着在响应时间和带宽使用方面的成本。它需要带宽使用来在应用服务器和 KDC 之间传输请求和响应。这可能会导致大容量应用程序服务器中出现一些性能问题。在这样的环境中，用户身份验证可能会导致额外的网络延迟和大量的流量。因此，默认情况下，KDC 不验证 PAC 签名。

5. PAC 在 kerberos 中的优缺点

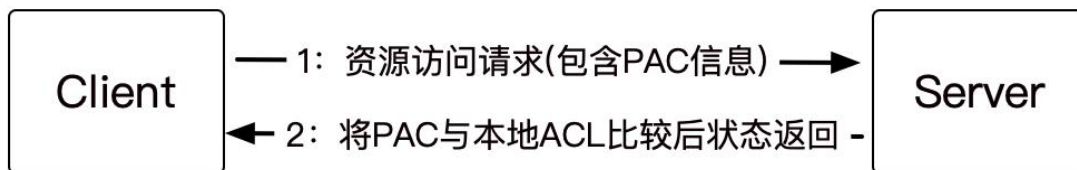
那么 PAC 的存在究竟给我们的验证过程带来了哪些优点，亦或是缺点呢？

正如上面所提到的那样，PAC 的引入其实带来了很多的优点。客户端在访问网络资源的时候服务端不再需要向 KDC 查询授权信息，而是直接在本地进行 PAC 信息与 ACL 的比较。从而节约了网络资源。

如图所示，在没有 PAC 的情况下，Server 与 KDC 之间必须进行用户授权信息的查询与返回：



当引入 PAC 之后则变成了如图所示：



但是，PAC 的引入并不是百利而无一害的，PAC 在用户的认证阶段引入会导致认证耗时过长。Windows Kerberos 客户端会通过 RPC 调用 KDC 上的函数来验证 PAC 信息，这时候用户会观察到在服务器端与 KDC 之间的 RPC 包流量的增加。而另一方面，由于 PAC 是微软特有的一个特性，所以启用了 PAC 的域中将不支持装有其他操作系统的服务器，制约了域配置的灵活性。并且在 2014 年，由于 PAC 的安全性导致产生了一个域内极其严重的提权漏洞 MS14-068(在后面的文章中我们会介绍这个漏洞)。

Kerberos 实验

为了更直观的分析 Kerberos 协议，接下来我们用普通域帐户 xie/hack 使用 impacket 工具请求 win10 机器的 cifs 服务票据，然后远程 SMB 连接，在该过程中使用 WireShark 进行抓包。

实验环境如下：

- 用户(xie/hack): 10.211.55.2
- 域内主机(Win10): 10.211.55.16
- 域控(AD01): 10.211.55.4

impacket 使用命令如下：

#使用 hack 账号密码请求 win10 的 cifs 服务的 ST 服务票据

```
python3 getST.py -dc-ip 10.211.55.4 -spn cifs/win10.xie.com xie.com/hack:P@ss1234
```

#导入该 ST 服务票据

```
export KRB5CCNAME=hack.ccache
```

#使用 smb 远程连接 win10

```
python3 smbexec.py -no-pass -k win10.xie.com
```


如图所示，可以看到在请求了服务票据后，成功远程 SMB 连接 win10 机器。

```
+ examples git:(master) * sudo python3 getST.py -dc-ip 10.211.55.4 -spn cifs/win10.xie.com xie.com/hack:P@ss1234
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

Using Kerberos Cache: hack@win10.xie.com.ccache
[*] Getting TGT for user
[*] Getting ST for user
[*] Saving ticket in hack.ccache
+ examples git:(master) * export KRB5CCNAME=hack.ccache
+ examples git:(master) * python3 smbexec.py -no-pass -k win10.xie.com
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

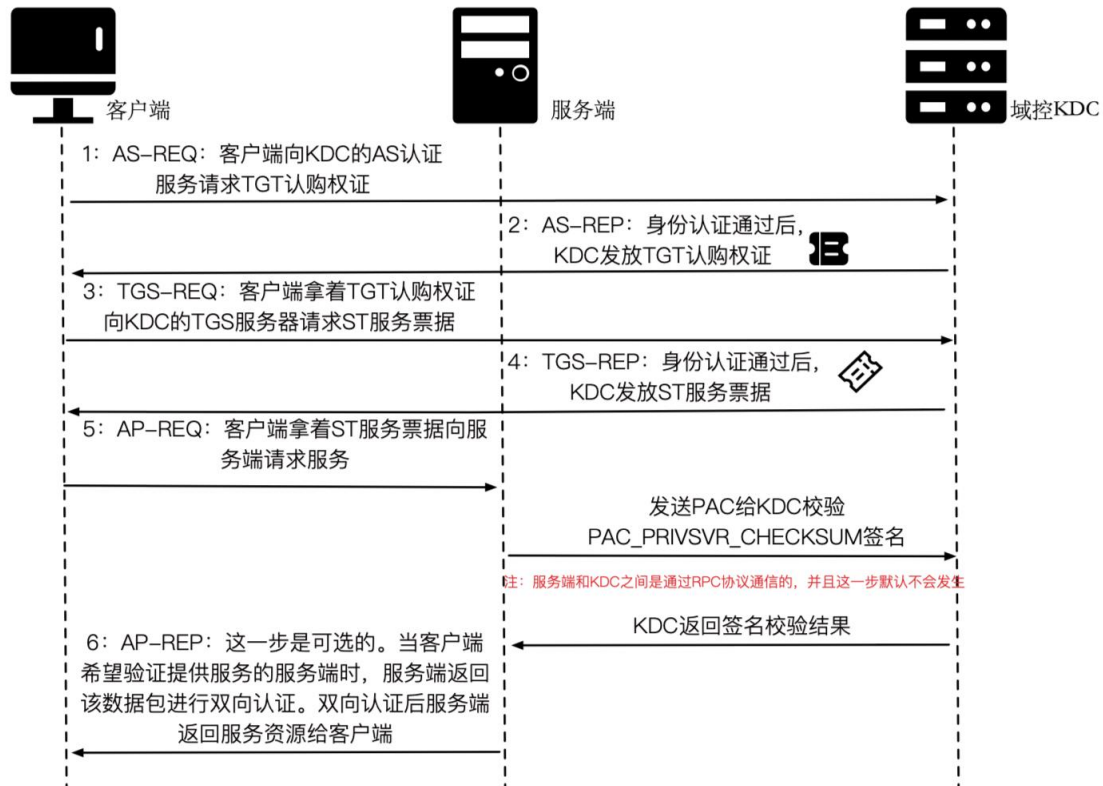
[!] Launching semi-interactive shell - Careful what you execute
C:\WINDOWS\system32>whoami
nt authority\system
```

在这个过程中，我们使用 WireShark 抓包，来进一步的分析 Kerberos 协议。如图所示，是该过程的抓包图：

10.211.55.2	10.211.55.4	KRB5	313	AS-REQ
10.211.55.4	10.211.55.2	KRB5	1425	AS-REP
10.211.55.2	10.211.55.4	KRB5	1327	TGS-REQ
10.211.55.4	10.211.55.2	KRB5	1356	TGS-REP
10.211.55.2	10.211.55.16	SMB2	1303	Session Setup Request

整个 Kerberos 认证流程如图所示：

Kerberos身份认证流程



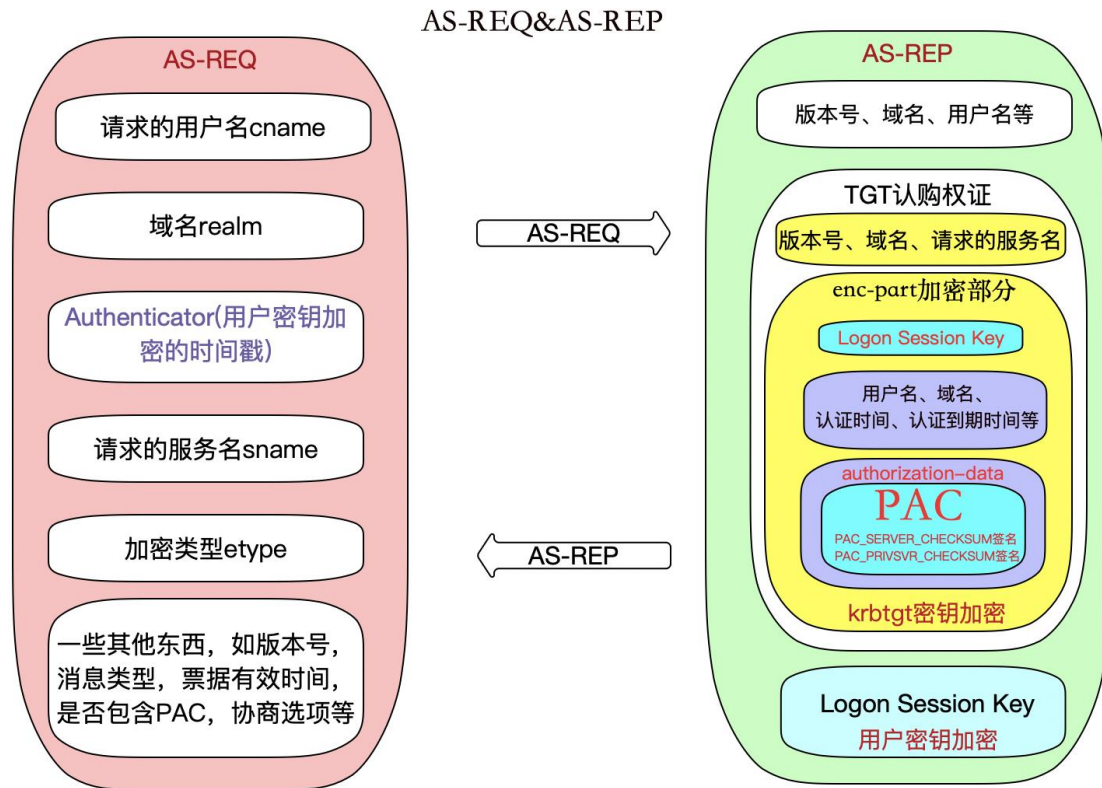
下面我们来具体分析 Kerberos 认证流程的每个步骤：

AS-REQ & AS-REP

我们先来看看 AS-REQ&AS-REP 请求部分，也就是 WireShark 抓的第一、二个包，如图所示：

10.211.55.2	10.211.55.4	KRB5	313	AS-REQ
10.211.55.4	10.211.55.2	KRB5	1425	AS-REP

如图所示，是一个简易的 AS-REQ&AS-REP 请求过程图，便于我们直观的了解 AS-REQ&AS-REP 请求过程。



下面让我们具体的分析下 AS-REQ&AS-REP 过程中的数据包细节：

首先，我们来看看客户端是如何获得 TGT 认购权证的。TGT 认购权证是由 KDC 的 AS (Authentication Service) 认证服务发放的。

1. AS-REQ 请求包分析

AS-REQ：当域内某个用户想要访问域内某个服务时，于是输入用户名和密码，本机就会向 KDC 的 AS 认证服务发送一个 AS-REQ 认证请求。该请求包中包含如下信息：

- 请求的用户名(cname)。
- 域名(realm)。
- Authenticator：一个抽象的概念，代表一个验证。这里是用户密钥加密的时间戳。

- 请求的服务名(sname): AS-REQ 这个阶段请求的服务都是 krbtgt。
- 加密类型(etype)。
- 以及一些其他信息: 如版本号, 消息类型, 票据有效时间, 是否包含 PAC, 协商选项等。

如图所示, 是 AS-REQ 请求包的详细:



```

Kerberos
  > Record Mark: 243 bytes
  > as-req
    pvno: 5
    msg-type: krb-as-req (10)
    > padata: 2 items
      > PA-DATA pA-ENC-TIMESTAMP
        > padata-type: pA-ENC-TIMESTAMP (2)
          > padata-value: 3041a003020112a23a04384545b0beb31066451323bb1bf701c0c68b9b966f221ef86230...
            etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
            cipher: 4545b0beb31066451323bb1bf701c0c68b9b966f221ef86230c1b2462c9df268cc0416e7...
          > PA-DATA pA-PAC-REQUEST
            > padata-type: pA-PAC-REQUEST (128)
              > padata-value: 3005a0030101ff
                include-pac: True
      > req-body
        Padding: 0
        > kdc-options: 50800000
          0... .... = reserved: False
          .1.. .... = forwardable: True
          ..0. .... = forwarded: False
          ...1 .... = proxyable: True
          .... 0... = proxy: False
          .... .0.. = allow-postdate: False
          .... ..0. = postdated: False
          .... ...0 = unused7: False
          1... .... = renewable: True
          .0.. .... = unused9: False
          ..0. .... = unused10: False
          ...0 .... = opt-hardware-auth: False
          .... 0... = unused12: False
          .... .0.. = unused13: False
          .... ..0. = constrained-delegation: False
          .... ...0 = canonicalize: False
          0... .... = request-anonymous: False
          .0.. .... = unused17: False
          ..0. .... = unused18: False
          ...0 .... = unused19: False
          .... 0... = unused20: False
          .... .0.. = unused21: False
          .... ..0. = unused22: False
          .... ...0 = unused23: False
          0... .... = unused24: False
          .0.. .... = unused25: False
          ..0. .... = disable-transited-check: False
          ...0 .... = renewable-ok: False
          .... 0... = enc-tkt-in-skey: False
          .... .0.. = unused29: False
          .... ..0. = renew: False
          .... ...0 = validate: False
        > cname
          name-type: kRB5-NT-PRINCIPAL (1)
          > cname-string: 1 item
            CNameString: hack
          realm: XIE.COM
        > sname
          name-type: kRB5-NT-PRINCIPAL (1)
          > sname-string: 2 items
            SNameString: krbtgt
            SNameString: XIE.COM
          till: 2021-12-21 11:48:16 (UTC)
          rtime: 2021-12-21 11:48:16 (UTC)
          nonce: 206793671
        > etype: 1 item
          ENCTYPE: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)

```

下面我们来看看 AS-REQ 请求包中每个字段的含义，如下所示：

pvno: kerberos 版本号,这里为 5

msg-type: 消息类型, AS_REQ 对应的是 krb-as-req(10)

pdata: 主要是一些认证信息, 每个认证消息有 type 和 value。

PA-DATA PA-ENC-TIMESTAMP: 这个是预认证, 就是用用户 Hash 加密时间戳, 作为 value 发送给 KDC 的 AS 服务。然后 KDC 从活动目录中查询出用户的 hash, 使用用户 Hash 进行解密, 获得时间戳, 如果能解密, 且时间戳在一定的范围内, 则证明认证通过。由于是使用用户密码 Hash 加密的时间戳, 所以也就造成了哈希传递攻击

pdata-type: pdata 类型,这里是 pA-ENC-TIMESTAMP(2)

pdata-value: 加密后的值

etype: 加密类型, 这里是 eTYPE-AES256-CTS-HMAC-SHA1-96(18)

cipher: 密钥

PA-DATA PA-PAC-REQUEST: 这个是启用 PAC 支持的扩展。这里的 value 对应的值为 True 或 False,KDC 根据 include 的值来确定返回的票据中是否需要携带 PAC。

pdata-type: pdata 类型,这里是 pA-PAC-REQUEST(128)

pdata-value: pdata 的值

include-pac: 是否包含 PAC,这里为 True,说明要 PAC

req-body: 请求 body

padding:填充,这里为 0

kdc-options:用于与 KDC 协商一些选项设置

reserved

forwardable

forwarded

proxiable

proxy

allow-postdate

postdated

unused7

renewable

unused9

unused10

opt-hardware-auth

unused12

unused13
 constrained-delegation
 canonicalize
 request-anonymous
 unused17
 unused18
 unused19
 unused20
 unused21
 unused22
 unused23
 unused24
 unused25
 disable-transited-check
 renewable-ok
 enc-~~tk~~-in-skey
 unused29
 renew
 validate

cname: 请求的用户名,这个用户名存在和不存在, 返回的包有差异, 因此可以用于枚举域内用户名。并且当用户名存在, 密码正确和错误时, 返回包也不一样, 因此也可以进行密码喷洒。

name-type:名字类型, 这里是 KRB5-NT-PRINCIPAL(1)

cnmae-string:名字, 也就是请求的用户名

CNameString: 请求的用户名, 这里为 hack

realm:域名, 这里为 XIE.COM

sname: 请求的服务, 包含 type 和 Value。在 AS-REQ 里面 sname 始终为 krbtgt

name-type:名字类型, 这里是 KRB5-NT-PRINCIPAL(1)

sname-string: krbtgt 用户的信息,这里有 2 个 items

SNameString: 这里是 krbtgt 用户名

SNameString: 这里是域名 XIE.COM

till:到期时间

rtime: 也是到期时间

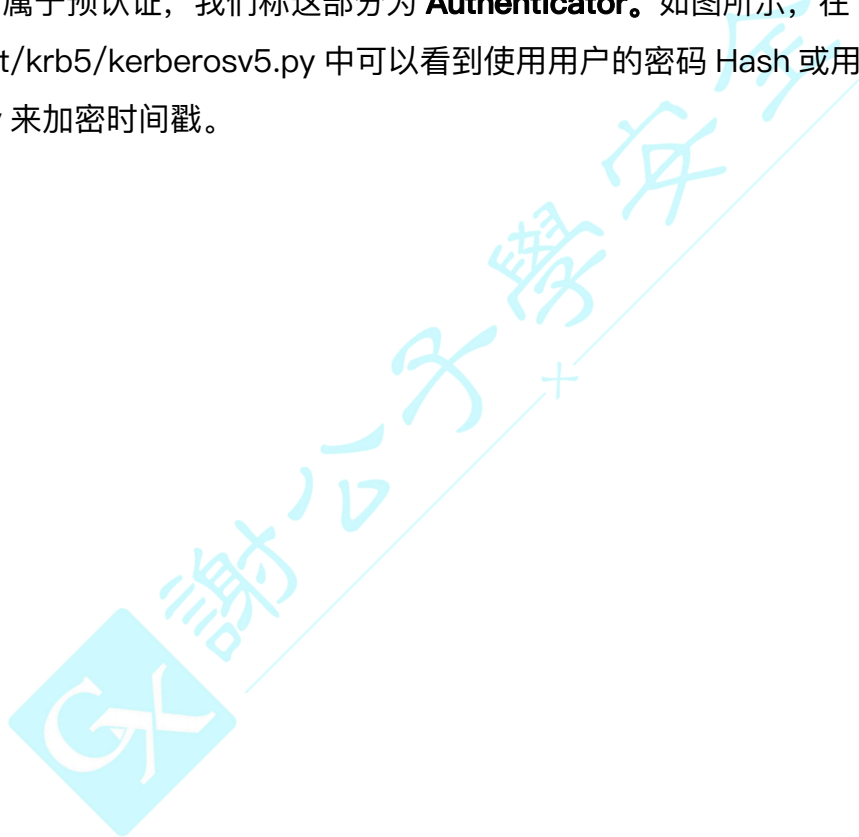
nonce: 随机生成的一个数

etype: 加密类型

ENCTYPE: eTYPE-AES256-CTS-HMAC-SHA1-96(18)

我们着重讲一下 PA-DATA PA-ENC-TIMESTAMP 字段, 该字段是用于预认证。

在 AS-REQ 请求包中, 只有 PA-DATA PA-ENC-TIMESTAMP 部分是加密的, 这一部分属于预认证, 我们称这部分为 **Authenticator**。如图所示, 在 `impacket/krb5/kerberosv5.py` 中可以看到使用用户的密码 Hash 或用户的密码 AES Key 来加密时间戳。



```

# Pass the hash/aes key :P
if nthash != b'' and (isinstance(nthash, bytes) and nthash != b''):
    key = Key(cipher.etype, nthash)
elif aesKey != b'':
    key = Key(cipher.etype, aesKey)
else:
    key = cipher.string_to_key(password, encryptionTypesData[etype], None)

if preAuth is True:
    if etype in encryptionTypesData is False:
        raise Exception('No Encryption Data Available!')

    # Let's build the timestamp
    timeStamp = PA_ENC_TS_ENC()

    now = datetime.datetime.utcnow()
    timeStamp['patimestamp'] = KerberosTime.to_asn1(now)
    timeStamp['pausage'] = now.microsecond

    # Encrypt the shyte
    encodedTimeStamp = encoder.encode(timeStamp)

    # Key Usage 1
    # AS-REQ PA-ENC-TIMESTAMP padata timestamp, encrypted with the
    # client key (Section 5.2.7.2)
    encryptedTimeStamp = cipher.encrypt(key, 1, encodedTimeStamp, None)

    encryptedData = EncryptedData()
    encryptedData['etype'] = cipher.etype
    encryptedData['cipher'] = encryptedTimeStamp
    encodedEncryptedData = encoder.encode(encryptedData)

```

对 WireShark 抓取的流量进行解密，如图所示，可以看到这里是使用 hack 用户的密钥来解密该值，如下的 patimestamp 和 pauses 是解密后的值。

```

PA-DATA pA-ENC-TIMESTAMP
  padata-type: pA-ENC-TIMESTAMP (2)
  padata-value: 3041a003020112a23a04384545b0beb31066451323bb1bf701c0c68b9b966f221ef86230...
    etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
  cipher: 4545b0beb31066451323bb1bf701c0c68b9b966f221ef86230c1b2462c9df268cc0416e7...
    > Decrypted keytype 18 usage 1 using keytab principal hack@xie.com (id=keytab.85 same=0) (2ca68074...)
      patimestamp: 2021-12-20 11:48:16 (UTC)
      pausec: 142604

```

2. AS-REP 回复包分析

AS-REP: 当 KDC 的 AS 认证服务接收到客户端发来的 AS-REQ 请求后, 从活动目录数据库中取出该用户的密钥, 然后用该密钥对请求包中的 Authenticator 预认证部分进行解密, 如果解密成功, 并且时间戳在有效的范围内, 则证明请求者提供的用户密钥正确。KDC 的 AS 认证服务在成功认证客户端的身份之后, 发送 AS-REP 响应包给客户端。AS-REP 响应包中主要包括如下信息:

- 请求的用户名(cname)。
- 域名(crealm)。
- TGT 认购权证: 包含明文的版本号, 域名, 请求的服务名, 以及加密部分 enc-part。加密部分用 krbtgt 密钥加密。加密部分包含 Logon Session Key、用户名、域名、认证时间、认证到期时间和 authorization-data。authorization-data 中包含最重要的 PAC 特权属性证书(包含用户的 RID, 用户所在组的 RID) 等。
- enc_Logon Session Key: 使用用户密钥加密 Logon Session Key 后的值, 其作用是用于确保客户端和 KDC 下阶段之间通信安全。也就是 AS-REP 中最外层的 enc-part。
- 以及一些其他信息: 如版本号, 消息类型等。

如图所示, 是 AS-REP 回复包的详细:

```

Kerberos
  > Record Mark: 1355 bytes
  > as-rep
    pvno: 5
    msg-type: krb-as-rep (11)
    > padata: 1 item
      > PA-DATA pA-ETYPE-INFO2
        > padata-type: pA-ETYPE-INFO2 (19)
          > padata-value: 30163014a003020112a10d1b0b5849452e434f4d6861636b
            > ETYPE-INFO2-ENTRY
              etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
              salt: XIE.COMhack
        crealm: XIE.COM
      > cname
        name-type: kRB5-NT-PRINCIPAL (1)
        > cname-string: 1 item
          CNameString: hack
    > ticket
      tkt-vno: 5
      realm: XIE.COM
      > sname
        name-type: kRB5-NT-PRINCIPAL (1)
        > sname-string: 2 items
          SNameString: krbtgt
          SNameString: XIE.COM
      > enc-part
        etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
        kvno: 2
        cipher: 72418e44eaa888e1a11d1d3499897bc0b314d559e58efc8a8fbab8ea52e4ff90be631e3f...
      > enc-part
        etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
        kvno: 3
        cipher: 57cdac04378000c4360b206ef62dbaa3cc28a39172cf8ff34c41814377581e479ce8f833...

```

下面我们来看看 AS-REP 回复包中每个字段的含义，如下所示：

pvno:kerberos 版本号,这里为 5

msg-type:消息类型,AS_REP 对应的是 krb-as-rep(11)

padata: 主要是一些认证信息。一个列表，包含若干个认证消息用于认证

PA-DATA PA-ENCTYPE-INFO2

padata-type: padata 的类型，这里是 pA-ETYPE-INFO2(19)

padata-value: 加密后的值

ETYPE-INFO2-ENTRY

etype: eTYPE-AES256-CTS-HMAC-SHA1-96(18)

salt: 盐值，这里是 XIE.COMhack

crealm: 域名,这里是 XIE.COM

cname: 请求的用户名

name-type: 用户名类型，这里是 kRB5-NT-PRINCIPAL(1)

cname-string: 1item
 CNameString: hack
 ticket: TGT 认购权证
 tgt-vno: TGT 版本号, 这里为 5
 realm: 域名, 这里是 XIE.COM
 sname: 服务用户名, 这里是 krbtgt 密码分发中心服务账号
 name-type: KRB5-NT-SRV-INST(2)
 sname-string: 2items
 SNameString: krbtgt
 SNameString: XIE.COM
 enc-part: TGT 票据中的加密部分, 这部分是用 krbtgt 的密码 Hash 加密的。因此如果我们拥有 krbtgt 的 hash 就可以自己制作一个 ticket, 这就造成了黄金票据攻击
 etype: 加密类型, 这里是 eTYPE-AES256-CTS-HMAC-SHA1-96(18)
 kvno: 版本号, 这里为 2
 cipher: 加密后的值
 enc-part: Login session key, 这部分是用请求的用户密码 Hash 加密的, 作为下阶段的认证密钥。
 etype: eTYPE-AES256-CTS-HMAC-SHA1-96(18)
 kvno: 版本号, 这里为 3
 cipher: 加密后的值

AS-REP 返回包中最重要的就是 TGT 认购权证和加密的 Logon Session Key 了。TGT 认购权证中加密部分是使用 krbtgt 密钥加密的, 而 Logon Session Key 是使用请求的用户密钥加密的。下面我们通过解密 WireShark 来看看 TGT 认购权证和 Logon Session Key 中到底包含哪些内容。

(1) TGT 认购权证

AS-REP 响应包中的 ticket 便是 TGT 认购权证了。TGT 认购权证中包含一些明文显示的信息, 如版本号 tgt-vno、域名 realm、请求的服务名 sname。但是 TGT 认购权证中最重要的还是加密部分, 加密部分是使用 krbtgt 帐户密钥加密的。加密部分主要包含的内容有 Logon Session Key、请求的用户名 cname、域名 crealm、认证时间 authtime、认证到期时间 endtime、authorization-data 等

权限

如图所示，是 TGT 认购权证：

```

ticket
  tkt-vno: 5
  realm: XIE.COM
  sname
    name-type: kRB5-NT-PRINCIPAL (1)
    sname-string: 2 items
      SNameString: krbtgt
      SNameString: XIE.COM
  enc-part
    etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
    kvno: 2
    cipher: 72418e44eaa888e1a11d1d3499897bc0b314d559e58efc8a8fbab8ea52e4ff90be631e3f...
    Decrypted keytype 18 usage 2 using keytab principal krbtgt@xie.com (id=keytab.19 same=0) (3434056c...)
    encTicketPart
      Padding: 0
      flags: 50e10000
      key
        Learnt encTicketPart_key keytype 18 (id=16.1) (6ed482eb...)
        keytype: 18
        keyvalue: 6ed482eb084249b10eff943583ac85378f040f1b81f3c17e885bc2ec928fe5c5
        realm: XIE.COM
      cname
        name-type: kRB5-NT-PRINCIPAL (1)
        cname-string: 1 item
          CNameString: hack
      transited
        tr-type: 0
        contents: <MISSING>
        authtime: 2021-12-20 11:48:16 (UTC)
        starttime: 2021-12-20 11:48:16 (UTC)
        endtime: 2021-12-20 21:48:16 (UTC)
        renew-till: 2021-12-21 11:48:16 (UTC)
  authorization-data: 1 item
    AuthorizationData item
      ad-type: ad-IF-RELEVANT (1)
      ad-data: 308202923082028ea0040202080a1820284048202800500000000000001000000b001...
      AuthorizationData item
        ad-type: ad-WIN2K-PAC (128)
        ad-data: 050000000000000001000000b00100005800000000000000a00000012000000802000...
          Num Entries: 5
          Version: 0
          Type: Logon Info (1)
          Type: Client Info Type (10)
          Type: UPN DNS Info (12)
          Type: Server Checksum (6)
          Type: Privsvr Checksum (7)

```

我们来看看 TGT 认购权证中 authorization-data 字段下代表用户身份权限的 PAC 是啥样的。我们对 PAC 进行解密，只查看 PAC 的凭证信息部分 PAC_LOGON_INFO。如图所示，最主要的还是通过 User RID 和 Group RID 来辨别用户权限的。

```

√ PAC_LOGON_INFO: 01100800ccccccca001000000000000000020091d2b94b97f5d701ffffffffffff7f...
√ MES header
  Version: 1
  √ DREP
    Byte order: Little-endian (1)
    HDR Length: 8
    Fill bytes: 0xcccccccc
    Blob Length: 416
  √ PAC_LOGON_INFO:
    Referent ID: 0x00020000
    Logon Time: Dec 20, 2021 19:46:59.005505700 CST
    Logoff Time: Infinity (absolute time)
    Kickoff Time: Infinity (absolute time)
    PWD Last Set: Jul 1, 2021 21:25:09.806508900 CST
    PWD Can Change: Jul 2, 2021 21:25:09.806508900 CST
    PWD Must Change: Infinity (absolute time)
  > Acct Name: hack
  > Full Name: hack
  > Logon Script
  > Profile Path
  > Home Dir
  > Dir Drive
  > Logon Count: 28
  > Bad PW Count: 0
  > User RID: 1154
  > Group RID: 513
  > Num RIDs: 1
  > GroupIDs
  > User Flags: 0x00000020
  > User Session Key: 00000000000000000000000000000000
  > Server: AD01
  > Domain: XIE
  > SID pointer:
    Dummy1 Long: 0x00000000
    Dummy2 Long: 0x00000000
  > User Account Control: 0x00000010
    Dummy4 Long: 0x00000000
    Dummy5 Long: 0x00000000
    Dummy6 Long: 0x00000000
    Dummy7 Long: 0x00000000
    Dummy8 Long: 0x00000000
    Dummy9 Long: 0x00000000
    Dummy10 Long: 0x00000000
  > Num Extra SID: 1
  > SID_AND_ATTRIBUTES_ARRAY:
  > ResourceGroupIDs

```

KDC 生成 PAC 的过程如下：KDC 在收到客户端发来的 AS-REQ 请求后，从请求中取出 cname 字段，然后查询活动目录数据库，找到 sAMAccountName 属性为 cname 字段的值的用户，用该用户的身份生成一个对应的 PAC。

(2) Logon Session Key

AS-REP 响应包最外层的那部分便是加密的 Login session Key 了，其作用是用于确保客户端和 KDC 下阶段之间通信安全，它使用请求的用户密钥加密。

我们对最外层的 enc-part 部分进行解密，如图所示，可以看到是使用 hack 用户的密钥对其进行解密的。

```

enc-part
  etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
  kvno: 3
  cipher: 57cdac04378000c4360b206ef62dbaa3cc28a39172cf8ff34c41814377581e479ce8f833...
  > Decrypted keytype 18 usage 3 using keytab principal hack@xie.com (id=keytab.85 same=0) (2ca68074...)
  encASRepPart
    key
      > Learnt encASRepPart_key keytype 18 (id=16.2) (6ed482eb...)
      keytype: 18
      keyvalue: 6ed482eb084249b10eff943583ac85378f040f1b81f3c17e885bc2ec928fe5c5
    last-req: 1 item
    LastReq item
      lr-type: LR-NONE (0)
      lr-value: 2021-12-20 11:48:16 (UTC)
    nonce: 206793671
    key-expiration: 2037-09-14 02:48:05 (UTC)
    Padding: 0
    flags: 50e10000
      0... .... = reserved: False
      .1... .... = forwardable: True
      ..0. .... = forwarded: False
      ...1 .... = proxiable: True
      .... 0... = proxy: False
      .... .0... = may-postdate: False
      .... ..0. = postdated: False
      .... ...0 = invalid: False
      1... .... = renewable: True
      .1... .... = initial: True
      ..1. .... = pre-authent: True
      ...0 .... = hw-authent: False
      .... 0... = transited-policy-checked: False
      .... .0... = ok-as-delegate: False
      .... ..0. = unused: False
      .... ...1 = enc-pa-rep: True
      0... .... = anonymous: False
    authtime: 2021-12-20 11:48:16 (UTC)
    starttime: 2021-12-20 11:48:16 (UTC)
    endtime: 2021-12-20 21:48:16 (UTC)
    renew-till: 2021-12-21 11:48:16 (UTC)
    srealm: XIE.COM
    sname
      name-type: KRB5-NT-PRINCIPAL (1)
      sname-string: 2 items
        SNameString: krbtgt
        SNameString: XIE.COM
  Provides learnt encTicketPart_key in frame 16 keytype 18 (id=16.1 same=0) (6ed482eb...)
  Provides learnt encASRepPart_key in frame 16 keytype 18 (id=16.2 same=0) (6ed482eb...)
  Used keytab principal krbtgt@xie.com keytype 18 (id=keytab.10 same=0) (3434056c...)

```

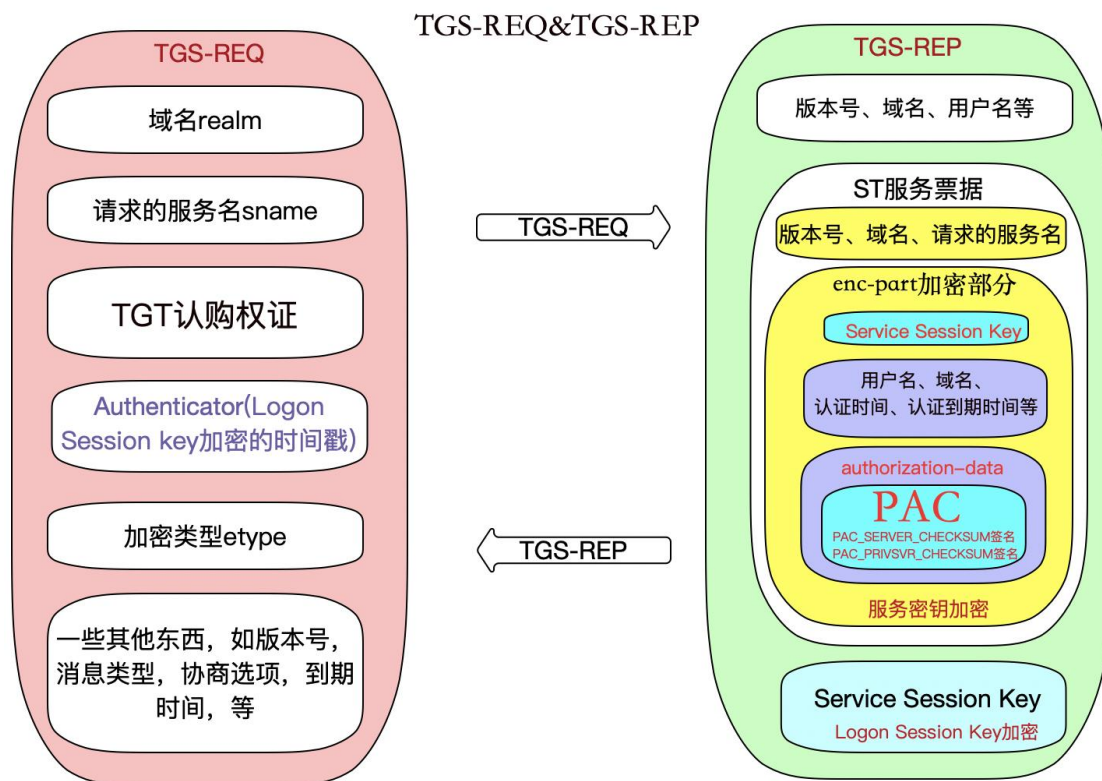
解密结果如下：主要包含的内容是认证时间 authtime、认证到期时间 endtime、域名 srealm、请求的服务名 sname、协商标志 flags 等一些信息。需要说明的是，在 TGT 认购权证中也包含 Logon Session Key。

TGS-REQ & TGS-REP

我们再来看看 TGS-REQ&TGS-REP 请求部分，也就是 WireShark 抓的第三、四个包，如图所示：

10.211.55.2	10.211.55.4	KRB5	1327	TGS-REQ
10.211.55.4	10.211.55.2	KRB5	1356	TGS-REP

如图所示，是一个简易的 TGS-REQ&TGS-REP 请求过程图，便于我们直观的了解 TGS-REQ&TGS-REP 请求过程。



下面让我们具体的分析下 TGS-REQ&TGS-REP 过程中的数据包细节：

客户端再收到 KDC 的 AS-REP 回复后，使用用户密钥解密 enc_Logon Session Key(也就是最外层的 enc-part)，得到 Logon Session Key，并且也拿到了 TGT 认购权证。之后它会在本地缓存此 TGT 认购权证 和 Logon Session Key。现在客户端需要凭借这张 TGT 认购凭证向 KDC 购买相应的 ST 服务票据（Service Ticket）。ST 服务票据是 KDC 的另一个服务 **TGS (Ticket Granting Service)** 票据授予服务发放的。在这个阶段，微软引入了两个扩展子协议 S4u2self 和 S4u2Proxy(当委派的时候，才用的到，我们会在后面的 4.5 章中详细介绍)。

1. TGS-REQ 请求包分析

TGS-REQ：客户端拿着上一步获得的 TGT 认购权证发起 TGS-REQ 请求，向 KDC 购买针对指定服务的 ST 服务票据，该请求主要包含如下信息：

- 域名(realm)。
- 请求的服务名(sname)。
- TGT 认购权证。
- Authenticator：一个抽象的概念，代表一个验证。这里使用 Logon Session Key 加密的时间戳。
- 加密类型(etype)。
- 以及一些其他信息：如版本号，消息类型，协商选项，票据到期时间等。

如图所示，是 TGS-REQ 请求包的详细：

```

Kerberos
  > Record Mark: 1257 bytes
  > tgs-req
    pvno: 5
    msg-type: krb-tgs-req (12)
    > padata: 1 item
      > PA-DATA pA-TGS-REQ
        > padata-type: pA-TGS-REQ (1)
          > padata-value: 6e82045530820451a003020105a10302010ea2070305000000000a38203d0618203cc30...
            > ap-req
              pvno: 5
              msg-type: krb-ap-req (14)
              Padding: 0
              > ap-options: 00000000
              > ticket
              > authenticator
    > req-body
      Padding: 0
      > kdc-options: 40810010
        0... .... = reserved: False
        .1... .... = forwardable: True
        ..0... .... = forwarded: False
        ...0 .... = proxiable: False
        .... 0... = proxy: False
        .... .0... = allow-postdate: False
        .... ..0. = postdated: False
        .... ...0 = unused7: False
        1... .... = renewable: True
        .0... .... = unused9: False
        ..0. .... = unused10: False
        ...0 .... = opt-hardware-auth: False
        .... 0... = unused12: False
        .... .0... = unused13: False
        .... ..0. = constrained-delegation: False
        .... ...1 = canonicalize: True
        0... .... = request-anonymous: False
        .0... .... = unused17: False
        ..0. .... = unused18: False
        ...0 .... = unused19: False
        .... 0... = unused20: False
        .... .0... = unused21: False
        .... ..0. = unused22: False
        .... ...0 = unused23: False
        0... .... = unused24: False
        .0... .... = unused25: False
        ..0. .... = disable-transited-check: False
        ...1 .... = renewable-ok: True
        .... 0... = enc-tgt-in-skey: False
        .... .0... = unused29: False
        .... ..0. = renew: False
        .... ...0 = validate: False
      realm: XIE.COM
      > sname
        name-type: KRB5-NT-SRV-INST (2)
        > sname-string: 2 items
          SNameString: cifs
          SNameString: win10.xie.com
        till: 2021-12-21 11:48:16 (UTC)
        nonce: 307440284
      > etype: 4 items
        ENCTYPE: eTYPE-ARCFOUR-HMAC-MD5 (23)
        ENCTYPE: eTYPE-DES3-CBC-SHA1 (16)
        ENCTYPE: eTYPE-DES-CBC-MD5 (3)
        ENCTYPE: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)

```

下面我们来看看 TGS-REQ 请求包中每个字段的含义，如下所示：

pvno:kerberos 版本号,这里为 5

msg-type:消息类型,TGS_REQ 对应的是 krb-tgs-req(12)

adata: adata 中包含 ap_req,这个是 TGS_REQ 必须携带的部分,这部分会携带 AS_REQ 里面获取到的 TGT 认购权证和使用原始的 Logon Session Key 加密的时间戳。还有可能会有 PA_FOR_USER, 类型是 S4U2SELF,是一个唯一的标识符, 该标识符指示用户的身份, 该标识符由用户名和域名组成。S4U2Proxy 必须扩展 PA_FOR_USER 结构, 指定服务代表某个用户去请求针对服务自身的 kerberos 服务票据。还有可能会有 PA_PAC_OPTIONS, 类型是 PA_PAC_OPTIONS。S4U2Proxy 必须扩展 PA-PAC-OPTIONS 结构。如果是基于资源的约束委派, 就需要指定 Resource-based Constrained Delegation 位。

adata-type: adata 类型, 这里是 pA-TGS-REQ(1)

adata-value: adata 的值

ap_req: 这个是 TGS_REQ 必须携带的部分

pvno:5

msg-type:krb-ap-req(14)

padding:0

ap-options:00000000

reserved: False

use-session-key: False

mutual-required: False

ticket AS-REP 响应包中返回的 TGT 认购权证

tko-vno:5

realm: XIE.COM

sname

name-type: kRB5-NT-SRV-PRINCIPAL(1)

sname-string:2 items

SNameString: krbtgt

SNameString: XIE.COM

enc-part

etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)

kvno: 2

cipher: 加密后的值

authenticator: 原始 Logon Session Key 加密的时间戳, 用于保证会话安全

etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)

cipher: 加密后的值

req-body: 请求 body

padding: 这里为 0

kdc-options: 用于与 KDC 约定一些选项设置

reserved

forwardable

forwarded

proxiable

proxy

allow-postdate

postdated

unused7

renewable

unused9

unused10

opt-hardware-auth

unused12

unused13

constrained-delegation

canonicalize

request-anonymous

unused17

unused18

unused19

unused20

unused21

unused22

unused23

```

unused24
unused25
disable-transited-check
renewable-ok
enc-tktin-skey
unused29
renew
validate

```

realm:域名, 这里为 XIE.COM

sname: 要请求的服务名

name-type: KRB5-NT-SRV-INST(2)

sname-string: 2 items

SNameString: cifs

SNameString: win10.xie.com

till:到期时间, rubeus 和 kekeo 都是 20370913024805Z, 这个可以作为特征来检测工具。

nonce: 随机生成的一个数。

etype: 加密类型

ENCTYPE: eTYPE-ARCFOUR-HMAC-MD5(23)

ENCTYPE: eTYPE-DES3-CBC-SHA1(16)

ENCTYPE: eTYPE-DES-CBC-MD5 (3)

ENCTYPE: eTYPE-AES256-CTS-HMAC-SHA1-96(18)

我们着重讲一下 ap-req 中的 authenticator 字段, 该字段主要用于后阶段的会话安全认证。

为了确保后阶段的会话安全, TGS-REQ 中 ap-req 中的 authenticator 字段的值是用上一步 AS-REP 中返回的 Logon Session Key 加密的时间戳, 如图所示:


```

  authenticator
    etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
  cipher: 256bb8bd6a62309d0d4c12f488ef1b698b257d33277233db8759163428ac49bde24ed382...
    > Decrypted keytype 18 usage 7 using learnt encTicketPart_key in frame 16 (id=16.1 same=2) (6ed482eb...)
  authenticator
    authenticator-vno: 5
    crealm: XIE.COM
    > cname
    cusec: 150975
    ctime: 2021-12-20 11:48:16 (UTC)

```

如图所示，在 impacket/krb5/kerberosv5.py 可以看到使用如下加密方式使用 Logon Session Key 加密的时间戳，

```

now = datetime.datetime.utcnow()
authenticator['cusec'] = now.microsecond
authenticator['ctime'] = KerberosTime.to_asn1(now)

encodedAuthenticator = encoder.encode(authenticator)

# Key Usage 7
# TGS-REQ PA-TGS-REQ padata AP-REQ Authenticator (includes
# TGS authenticator subkey), encrypted with the TGS session
# key (Section 5.5.1)
encryptedEncodedAuthenticator = cipher.encrypt(sessionKey, 7, encodedAuthenticator, None)

apReq['authenticator'] = noValue
apReq['authenticator']['etype'] = cipher.etype
apReq['authenticator']['cipher'] = encryptedEncodedAuthenticator

```

AS-REP 中返回的 sessionKey

2. TGS-REP 回复包分析

TGS-REP: KDC 的 TGS 服务接收到 TGS-REQ 请求之后。首先使用 krbtgt 密钥解密 TGT 认购权证中加密部分得到 Logon Session key 和 PAC 等信息，如果能解密成功则说明该 TGT 认购权证是 KDC 颁发的。然后验证 PAC 的签名，如果签名正确，则证明 PAC 未经篡改。然后使用 Logon Session Key 解密 Authenticator 得到时间戳等信息，如果能够解密成功，并且票据时间在范围内，则验证了会话的安全性。在完成上述的检测后，KDC 的 TGS 服务完成了对客户端的认证，TGS 服务发送响应包给客户端。响应包中主要包括如下信息：

- 请求的用户名(cname)
- 域名(crealm)

- ST 服务票据：包含明文的版本号，域名，请求的服务名，以及加密部分 enc-part，加密部分用服务密钥加密。加密部分包含用户名、域名、认证时间、认证到期时间、Service Session key 和 authorization-data。authorization-data 中包含最重要的 PAC 特权属性证书(包含用户的 RID，用户所在的组的 RDI) 等。
- enc_Service Session key：使用 Logon Session key 加密的 Service Session key，其作用是用于确保客户端和 KDC 下阶段之间通信安全。
- 以及一些其他信息：如版本号、消息类型等。

注：这里需要说明的是，TGS-REP 这步中 KDC 并不会验证客户端是否有权限访问服务端。因此，这一步不管用户有没有访问服务的权限，只要 TGT 正确，均会返回 ST 服务票据，这也是 kerberoasting 能利用的原因，任何一个域内用户，都可以请求域内任何一个服务的 ST 服务票据。

如图所示，是 TGS-REP 回复包的详细：

```

Kerberos
  > Record Mark: 1286 bytes
  > tgs-rep
    pvno: 5
    msg-type: krb-tgs-rep (13)
    crealm: XIE.COM
    > cname
      name-type: kRB5-NT-PRINCIPAL (1)
      > cname-string: 1 item
        CNameString: hack
    > ticket
      tkt-vno: 5
      realm: XIE.COM
      > sname
        name-type: kRB5-NT-SRV-INST (2)
        > sname-string: 2 items
          SNameString: cifs
          SNameString: win10.xie.com
      > enc-part
        etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
        kvno: 2
        cipher: fd4e8fa746d58624949f70c320669b91484a463e471bbd72a432bd8680408aef90db0d94...
    > enc-part
      etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
      cipher: f468746f6ff848ca2fadfa1089153bd3bcc530bc606af27bd29cf619116295d3d01bc491...

```

下面我们来看看 TGS-REP 回复包中每个字段的含义，如下所示：

pvno:kerberos 版本号,这里为 5

msg-type:消息类型,TGS_REP 对应的是 krb-tgs-rep(13)

crealm: 域名,这里是 XIE.COM

cname: 请求的用户名

name-type: 名称类型, 这里为 KRB5-NT-PRINCIPAL(1)

cname-string: 1 item

CNameString: hack

ticket: 即 ST 服务票据

tko-vno: 服务票据版本号, 这里为 5

realm: 域名,这里是 XIE.COM

sname:

name-type: KRB5-NT-SRV-HST(3)

sname-string: 2 items

SNameString: cifs

SNameString: win10.xie.com

enc-part: 这部分是用服务的密钥加密的

etype: 加密类型, eTYPE-AES256-CTS-HMAC-SHA1-96(18)

kvno: 版本号, 这里为 3

cipher: 加密后的值

enc-part: 这部分是用原始的 Logon Session Key 加密的。里面最重要的字段是 Service session key, 作为下阶段的认证密钥。

etype: 加密类型 eTYPE-AES256-CTS-HMAC-SHA1-96(18)

cipher: 加密后的值

TGS-REP 返回包中最重要的就是 ST 服务票据和 Service Session key 了。ST 服务票据中加密部分是使用服务密钥加密的, 而 Service Session key 是使用 Logon Session Key 加密的。下面我们通过解密 WireShark 来看看 ST 服务票据和 Service Session key 中到底包含哪些内容。

(1) ST 服务票据

TGS-REP 响应包中的 ticket 便是 ST 服务票据了。ST 服务票据中包含明文显示的信息, 如版本号 tko-vno、域名 realm、请求的服务名 sname。但是 ST 服务票

据中最重要的还是加密部分，加密部分是使用服务密钥加密的。加密部分主要包含的内容有 Server Session Key、请求的用户名 cname、域名 crealm、认证时间 authtime、认证到期时间 endtime、authorization-data 等信息。最重要的还是 authorization-data 部分，这部分中包含客户端的身份权限等信息，这些信息包含在 PAC 中。

如图所示，是 ST 服务票据：

```

ticket
  tkt-vno: 5
  realm: XIE.COM
  sname
    name-type: kRB5-NT-SRV-INST (2)
    sname-string: 2 items
      SNameString: cifs
      SNameString: win10.xie.com
  enc-part
    etype: eTYPE-ARCFOUR-HMAC-MD5 (23)
    kvno: 5
    cipher: a431c0d3e3b029fff5d22a71786cd1ddc6ad19885349444294671c7689d03af25282382a...
    > Decrypted keytype 23 usage 2 using keytab principal cifs/win10.xie.com@xie.com (id=keytab.96 same=0) (74520a4e...)
    encTicketPart
      Padding: 0
      > flags: 40a10000
      > key
        crealm: XIE.COM
        > cname
        > transited
        authtime: 2021-12-21 09:33:34 (UTC)
        starttime: 2021-12-21 09:33:34 (UTC)
        endtime: 2021-12-21 19:33:34 (UTC)
        renew-till: 2021-12-22 09:33:34 (UTC)
      > authorization-data: 1 item
        > AuthorizationData item
          ad-type: aD-IF-RELEVANT (1)
          > ad-data: 308202ba308202b6a00402020080a18202ac048202a805000000000000001000000b001...
            > AuthorizationData item
              ad-type: aD-WIN2K-PAC (128)
              > ad-data: 0500000000000000001000000b00100005800000000000000a0000001200000008020000...
                Num Entries: 5
                Version: 0
                > Type: Logon Info (1)
                > Type: Client Info Type (10)
                > Type: UPN DNS Info (12)
                > Type: Server Checksum (6)
                > Type: Privsvr Checksum (7)

```

我们来看看 ST 服务票据中 authorization-data 字段下代表用户身份权限的 PAC 是啥样的。我们对 PAC 进行解密，只查看 PAC 的凭证信息部分 PAC_LOGON_INFO。如图所示，最主要的还是通过 User RID 和 Group RID 来辨别用户权限的。可以看到，ST 服务票据中的 PAC 和 TGT 认购权证中的 PAC 是一致的。在正常的非 S4u2Self 请求的 TGS 过程中，KDC 在 ST 服务票据中的 PAC 是直接拷贝 TGT 票据中的 PAC。

```

✓ PAC_LOGON_INFO: 01100800ccccccca001000000000000000000200e13cf5a44df6d701ffffffffffffffffff7f...
  > MES header
  ✓ PAC_LOGON_INFO:
    Referent ID: 0x00020000
    Logon Time: Dec 21, 2021 17:32:17.116899300 CST
    Logoff Time: Infinity (absolute time)
    Kickoff Time: Infinity (absolute time)
    PWD Last Set: Dec 21, 2021 17:27:11.531645200 CST
    PWD Can Change: Dec 22, 2021 17:27:11.531645200 CST
    PWD Must Change: Infinity (absolute time)
  > Acct Name: hack
  > Full Name: hack
  > Logon Script
  > Profile Path
  > Home Dir
  > Dir Drive
    Logon Count: 35
    Bad PW Count: 0
    User RID: 1154
    Group RID: 513
    Num RIDs: 1
  > GroupIDs
  > User Flags: 0x00000020
    User Session Key: 00000000000000000000000000000000
  > Server: AD01
  > Domain: XIE
  > SID pointer:
    Dummy1 Long: 0x00000000
    Dummy2 Long: 0x00000000
  > User Account Control: 0x00000010
    Dummy4 Long: 0x00000000
    Dummy5 Long: 0x00000000
    Dummy6 Long: 0x00000000
    Dummy7 Long: 0x00000000
    Dummy8 Long: 0x00000000
    Dummy9 Long: 0x00000000
    Dummy10 Long: 0x00000000
    Num Extra SID: 1
  > SID_AND_ATTRIBUTES_ARRAY:
  > ResourceGroupIDs

```



(2) Service Session Key

TGS-REP 响应包最外层的那部分便是 Service Session Key 了，其作用是用于确保客户端和 KDC 下阶段之间通信安全，它使用 Logon Session Key 加密。

如图所示，对其进行解密，它主要包含的内容是认证时间 `authtime`、认证到期时间 `endtime`、域名 `srealm`、请求的服务名 `sname`、协商标志 `flags` 等一些信息。需要说明的是，在 ST 服务票据中也包含 `Service Session Key`。

```

v enc-part
  etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
  cipher: f468746ff848ca2fadfa1089153bd3bcc530bc606af27bd29cf619116295d3d01bc491...
  Decrypted keytype 18 usage 8 using learnt encTicketPart_key in frame 16 (id=16.1 same=2) (6ed482eb...)
  encTGSRepPart
    > key
      > last-req: 1 item
      nonce: 307440284
      Padding: 0
      > flags: 40a10000
      authtime: 2021-12-20 11:48:16 (UTC)
      starttime: 2021-12-20 11:48:16 (UTC)
      endtime: 2021-12-20 21:48:16 (UTC)
      renew-til: 2021-12-21 11:48:16 (UTC)
      srealm: XIE.COM
    > sname
      name-type: kRB5-NT-SRV-INST (2)
      > sname-string: 2 items
        SNameString: cifs
        SNameString: win10.xie.com
    > encrypted-pa-data: 1 item
      > PA-DATA pa-SUPPORTED-ETYPES
        > padata-type: pa-SUPPORTED-ETYPES (165)
          > padata-value: 1f000000
            > SupportedEncTypes: 0x0000001f, des-cbc-crc, des-cbc-md5, rc4-hmac, aes128-cts-hmac-sha1-96, aes256-cts-hmac-sha1-96
              ....1 = des-cbc-crc: Supported
              ....1. = des-cbc-md5: Supported
              ....1.. = rc4-hmac: Supported
              ....1... = aes128-cts-hmac-sha1-96: Supported
              ....1.... = aes256-cts-hmac-sha1-96: Supported
              ....0 = fast-supported: Not supported
              ....0. = compound-identity-supported: Not supported
              ....0.. = claims-supported: Not supported
              ....0... = resource-sid-compression-disabled: Not supported

```

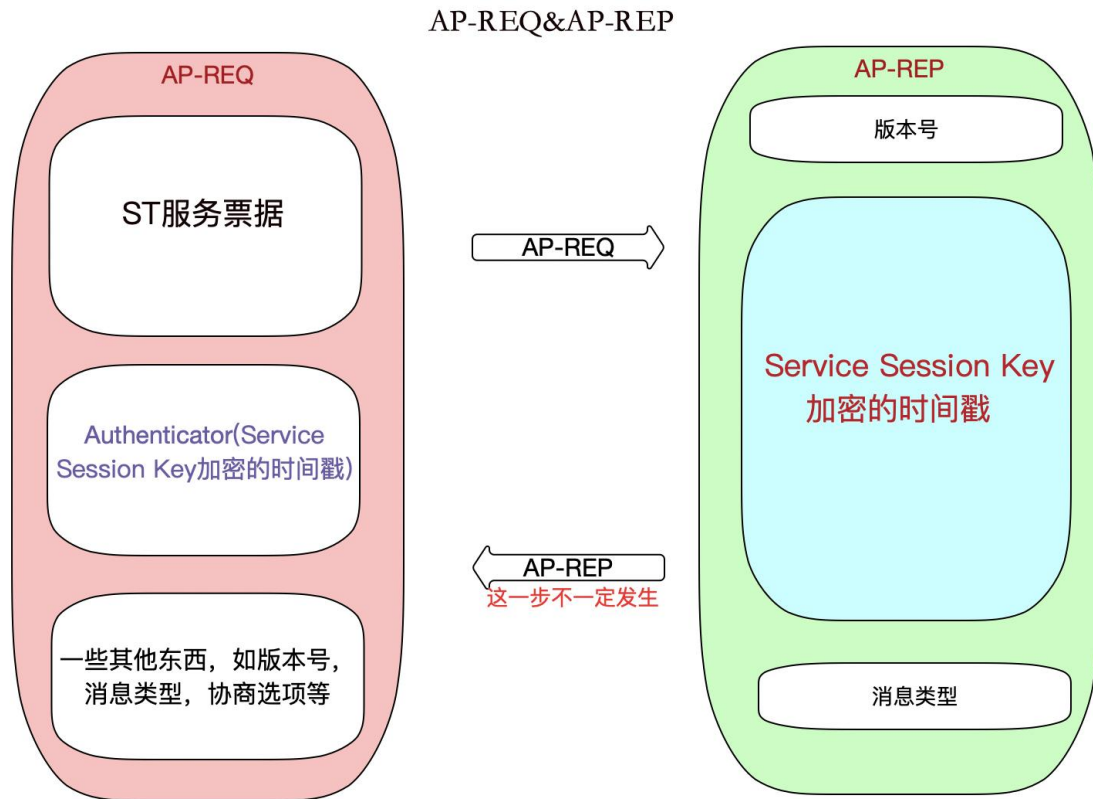
如何双向认证? (AP-REQ&AP-REP)

客户端在收到 KDC 返回的 TGS-REP 消息，从中取出 ST 服务票据后，就准备要开始申请访问服务了。由于我们是通过 SMB 协议远程连接的，因此 AP-REQ&AP-REP 消息是放在 SMB 协议中。如图所示：

kerberos&smb2						
	Time	Source	Destination	Protocol	Length	Info
40	18.942151	10.211.55.2	10.211.55.16	SMB2	1303	Session Setup Request

注：通过 impacket 远程连接服务默认是不需要验证提供服务的服务端的，因此这里没有 AP-REP 回复。

如图所示，是一个简易的 AP-REQ&AP-REP 请求过程图，便于我们直观的了解 AP-REQ&AP-REP 请求过程。



1. AP-REQ 请求包分析

AP-REQ: 客户端接收到 KDC 的 TGS 回复后, 通过缓存的 Logon Session Key 解密 enc_Service Session key 得到 Service Session Key, 同时它也拿到了 ST(Service Ticket)服务票据。Service Session Key 和 ST 服务票据会被客户端缓存。客户端访问指定服务时, 将发起 AP-REQ 请求, 该请求主要包含如下的内容:

- ST 服务票据(ticket)
- Authenticator: 一个抽象的概念, 代表一个验证。这里指 Service Session Key 加密的时间戳
- 以及一些其他信息: 如版本号、消息类型, 协商选项等

如图所示, 是 AP-REQ 请求包的详细:


```

  Security Blob: 60820ad306062b0601050502a0820ac730820ac3a030302e06092a864882f71201020206...
  GSS-API Generic Security Service Application Program Interface
    OID: 1.3.6.1.5.5.2 (SPNEGO - Simple Protected Negotiation)
  Simple Protected Negotiation
    negTokenInit
      mechTypes: 4 items
      mechToken: 60820a8506092a864886f71201020201006e820a7430820a70a003020105a10302010ea2...
      krb5_blob: 60820a8506092a864886f71201020201006e820a7430820a70a003020105a10302010ea2...
        KRB5 OID: 1.2.840.113554.1.2.2 (KRB5 - Kerberos 5)
        krb5_tok_id: KRB5_AP_REQ (0x0001)
  Kerberos
    ap-req
      pvno: 5
      msg-type: krb-ap-req (14)
      Padding: 0
      ap-options: 20000000
        0... .... = reserved: False
        .0.. .... = use-session-key: False
        ..1. .... = mutual-required: True
      ticket
        tkt-vno: 5
        realm: XIE.COM
        sname
          name-type: KRB5-NT-SRV-INST (2)
          sname-string: 2 items
            SNameString: cifs
            SNameString: Server2008.xie.com
        enc-part
          etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
          kvno: 3
          cipher: 81dd27704c5b34f6d73b8d0bd2fa974998fb8a137b2144825a65428381005026363c2b79...
        authenticator
          etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
          cipher: ef55061ec8a216b1a76af6fd924422f118eb97f8262309f0c39b34df43c5aef72db7e...

```

下面我们来看看 AP-REQ 请求包中每个字段的含义，如下所示：

pvno:kerberos 版本号,这里为 5

msg-type:消息类型, AP_REQ 对应的是 krb-ap-req(14)

Padding:填充,这里为 0

ap-options: 一些协商选项

reserved

use-session-key

mutual-required 该选项代表客户端是否希望验证提供服务的服务端

ticket: ST 服务票据

tkt-vno: 版本号, 这里为 5

realm: XIE.COM

sname:服务名

name-type: KRB5-NT-SRV-INST(2)

sname-string: 2 item2

SNameString: cifs
SNameString: win10.xie.com
enc-part: ST 服务票据中加密部分
etype:加密类型, eTYPE-AES256-CTS-HMAC-SHA1-96(18)
kvno: 版本号, 这里是 2
cipher:加密后的值
authenticator: Service Session Key 加密的时间戳
etype: 加密类型
cipher: 加密后的值

2. AP-REP 回复包分析

AP-REP: 这一步是可选的, 当客户端希望验证提供服务的服务端时(也就是 AP-REQ 请求中 mutual-required 协商选项为 True), 服务端返回 AP-REP 消息。服务端收到客户端发来的 AP-REQ 消息后, 通过服务密钥解密 ST 服务票据得到 Service Session Key 和 PAC 等信息, 然后用 Service Session Key 解密 Authenticator 得到时间戳。如果能解密成功且时间戳在有效范围内, 则验证了客户端的身份。验证了客户端身份后, 服务端从 ST 服务票据中取出 PAC 中代表用户身份权限信息的数据, 然后与请求的服务 ACL 做对比, 生成相应的访问令牌。同时, 服务端会检查 AP-REQ 请求中 mutual-required 协商选项是否为 True, 如果为 True 的话, 说明客户端想验证服务端的身份。此时, 服务端会用 Service Session Key 加密时间戳作为 Authenticator, 在 AP-REP 响应包中发送给客户端进行验证。如果 mutual-required 选项为 False 的话, 服务端会根据访问令牌的权限决定是否返回相应的服务给客户端。

注: 由于 impacket 默认是不需要验证服务端身份的, 因此如图所示是其他请求方式的 AP-REP 回复包截图。

SMB2 (Server Message Block Protocol version 2)
 > SMB2 Header
 > Session Setup Response (0x01)
 [Preauth Hash: 66a555b3e9f8b5a4e471fc6c0b2d3f3e30835388d5886d0e37a9a9da02a728fdd233891d...]
 > StructureSize: 0x0009
 > Session Flags: 0x0000
 Blob Offset: 0x00000048
 Blob Length: 184
 > Security Blob: a181b53081b2a0030a0100a10b06092a864882f712010202a2819d04819a60819706092a...
 > GSS-API Generic Security Service Application Program Interface
 > Simple Protected Negotiation
 > negTokenTarg
 negResult: accept-completed (0)
 supportedMech: 1.2.840.48018.1.2.2 (MS KRB5 - Microsoft Kerberos 5)
 responseToken: 60819706092a864886f71201020202006f8187308184a003020105a10302010fa2783076...
 > krb5_blob: 60819706092a864886f71201020202006f8187308184a003020105a10302010fa2783076...
 KRB5 OID: 1.2.840.113554.1.2.2 (KRB5 - Kerberos 5)
 krb5_tok_id: KRB5_AP_REP (0x0002)
 > Kerberos
 > ap-rep
 pvno: 5
 msg-type: krb-ap-rep (15)
 > enc-part
 etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
 cipher: e255adeb09ad742bc9ebef69df6b85a990aa98bf180f8e8ef8ecec5da5ce9c93d91af50...

AP-REP 响应包中主要包括如下信息：

- 版本号
- 消息类型
- enc-part：使用 Service Session Key 加密的时间戳

如下是 AP-REP 回复包的详细：

pvno: 5

msg-type: 消息类型, AP-REP 对应的是 krb-ap-rep(15)

enc-part: Service Session Key 加密的时间戳

etype: 加密类型

cipher: 加密后的值

S4u2Self&S4u2Proxy

为了在 Kerberos 协议层面对约束性委派的支持，微软对 Kerberos 协议扩展了两个自协议 S4u2self(Service for User to Self) 和 S4u2Proxy (Service for User to Proxy)。S4u2self 可以代表任意用户请求针对自身的服务票据；S4u2Proxy

可以用上一步获得的 ST 服务票据以用户的名义请求针对其它指定服务的 ST 服务票据。

执行如下命令，machine\$机器用户模拟 administrator 身份访问自身服务。

```
python3 getST.py -dc-ip AD01.xie.com xie.com/machine\$:root -spn cifs/ad01.xie.com -impersonate administrator
```

如图所示，machine\$机器账号以 S4u2Self 协议模拟 administrator 身份访问自身服务。

```
+ examples git:(master) * sudo python3 getST.py -dc-ip AD01.xie.com xie.com/machine\$:root -spn cifs/ad01.xie.com -impersonate administrator
Password:
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

Using Kerberos Cache: administrator.ccache
[*] Getting TGT for user
[*] Impersonating administrator
[*] Requesting S4u2self
forwardable标志位: 1
[*] Requesting S4u2Proxy
[*] Saving ticket in administrator.ccache
```

1. S4u2Self

和正常的 TGS-REQ 请求包相比，S4u2Self 协议的 TGS-REQ 请求包会多一个 PA-DATA pA-TGS-USER，name 为要模拟的用户。并且 sname 也是请求的服务自身。如图所示：

```

tgs-req
  pvno: 5
  msg-type: krb-tgs-req (12)
  padata: 2 items
    > PA-DATA pA-TGS-REQ
    > PA-DATA pA-FOR-USER
      padata-type: pA-FOR-USER (129) S4u2Self
      padata-value: 3051a01a3018a003020101a111300f1b0d61646d696e6973747261746f72a1091b077869...
        name
          name-type: kRB5-NT-PRINCIPAL (1)
          name-string: 1 item
            KerberosString: administrator
            realm: xie.com 要模拟的用户
        cksum
          cksumtype: cKSUMTYPE-HMAC-MD5 (-138)
          checksum: 558cda470cdf8898773dd815ef5e482
          auth: Kerberos
    > req-body
      Padding: 0
      kdc-options: 40810000
      realm: XIE.COM
      sname
        name-type: kRB5-NT-UNKNOWN (0)
        sname-string: 1 item
          SNameString: machine$ 服务自身
      till: 2021-12-27 13:08:26 (UTC)
      nonce: 1196166983
    > etype: 2 items

```

2. S4u2Proxy

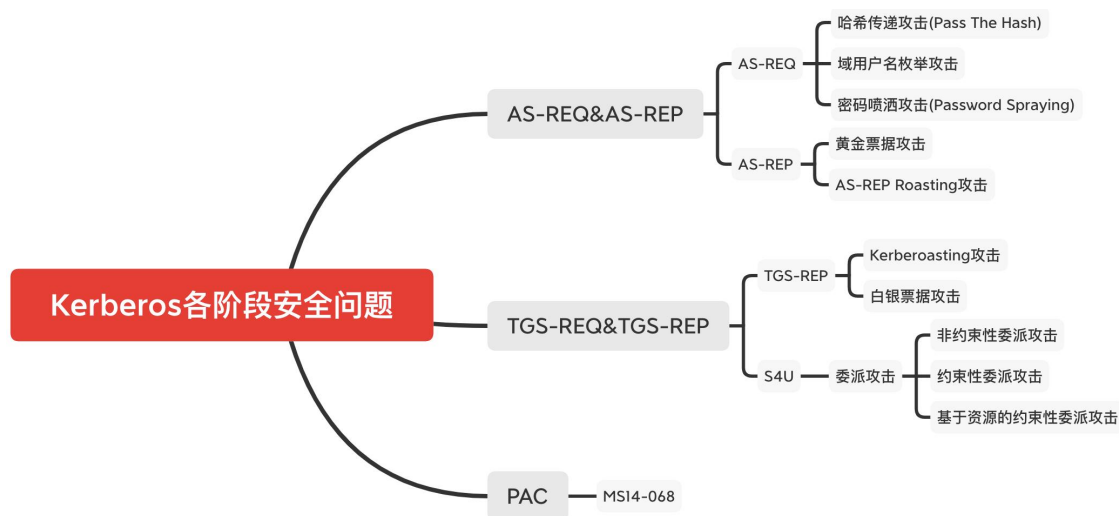
和正常的 TGS-REQ 请求包相比，S4u2Proxy 协议的 TGS-REQ 请求包会增加一个 additional-tickets 字段，该字段的内容就是上一步利用 S4u2Self 请求的 ST 服务票据。如图所示：

▼ Kerberos

- > Record Mark: 2272 bytes
- ▼ tgs-req
 - pvno: 5
 - msg-type: krb-tgs-req (12)
 - > padata: 1 item
 - ▼ req-body
 - Padding: 0
 - > kdc-options: 40820010
 - realm: XIE.COM
 - > sname
 - till: 2037-09-13 02:48:05 (UTC)
 - nonce: 1802073961
 - > etype: 3 items
 - ▼ additional-tickets: 1 item
 - ▼ Ticket **S4u2Self 请求的 ST 服务票据**
 - tkit-vno: 5
 - realm: XIE.COM
 - ▼ sname
 - name-type: kRB5-NT-PRINCIPAL (1)
 - > sname-string: 1 item
 - > enc-part

Kerberos 协议的安全问题

如图所示，是 Kerberos 协议各阶段容易产生的安全问题：



在 AS-REQ 请求阶段，是用用户密码 Hash 或 AES Key 加密的时间戳。因此当只获得了用户密码 Hash 时，也可以发起 AS-REQ 请求，所以也就造成了 **PTH 哈希传递攻击**；当只获得用户密码的 AES Key 时，也可以发起 AS-REQ 请求，所以也就造成了 **PTK 密钥传递攻击**。

而 AS-REQ 请求包中 cname 字段的值代表用户名，这个值存在和不存在，返回的包有差异，所以可以用于枚举域内用户名，这种攻击方式被称为 **域内用户枚举攻击**（当未获取到有效域用户权限时，可以使用这个方法枚举域内用户）。并且当用户名存在，密码正确和密码错误时，返回的包也不一样，所以可以进行用户名密码爆破。但是在实战中，渗透测试人员通常都会使用一种被称为 **密码喷洒**

(Password Spraying) 的攻击方式来进行测试和攻击。对密码进行喷洒式的攻击，这个叫法很形象，因为它属于自动化密码猜测的一种。这种针对所有用户的自动密码喷洒通常是为了避免帐户被锁定，因为针对同一个用户的连续密码猜测会导致帐户被锁定。所以只有对所有用户同时执行特定的密码登录尝试，才能增加破解的概率，消除帐户被锁定的概率。普通的爆破就是用户名固定，爆破密码，但是密码喷洒是用固定的密码去跑所有的用户名。

在 AS-REP 阶段，由于返回的 TGT 认购权证是由 krbtgt 用户的密码 Hash 加密的，因此如果我们拥有 krbtgt 的密码 hash 就可以自己制作一个 TGT 认购权证，这种攻击方式被称为**黄金票据攻击**。同样，在 TGS-REP 阶段，TGS-REP 里

面的 ST 服务票据是使用服务的 hash 进行加密的，如果我们拥有服务的 hash，就可以签发任意用户的 ST 服务票据，这个票据也被称为白银票据，这种攻击方式被称为**白银票据攻击**。相较于黄金票据，白银票据使用要访问服务的 hash，而不是 krbtgt 的 hash。

在 AS-REP 阶段，Login session key 是用用户密码 Hash 加密的。对于域用户，如果设置了“Do not require Kerberos preauthentication”不需要预认证选项，此时攻击者向域控制器的 88 端口发送 AS_REQ 请求，此时域控不会做任何验证就将 TGT 认购权证 和 该用户 Hash 加密的 Login Session Key 返回。因此，攻击者就可以对获取到的 用户 Hash 加密的 Login Session Key 进行离线破解，如果破解成功，就能得到该用户的密码明文，这种攻击方式被称为 **AS-REP Roasting 攻击**。

在 TGS-REP 阶段，由于 ST 服务票据是用服务 Hash 加密的。因此，如果我们能获取到 ST 服务票据，就可以对该 ST 服务票据进行离线破解，得到服务的 Hash，这种攻击方式被称为 **Kerberoasting 攻击**。这个问题存在的另外一个因素是因为用户向 KDC 发起 TGS_REQ 请求，不管用户对服务有没有访问权限，只要 TGT 认购权证正确，那么 KDC 都会返回 ST 服务票据。其实 AS_REQ 里面的服务就是 krbtgt，也就是说这个攻击方式同样可以用于爆破 AS-REP 里面的 TGT 认购权证，但是之所以没见到这种攻击方式是因为 krbtgt 的密码是随机生成的，爆破不出来。

非常感谢您读到现在，由于作者的水平有限，编写时间仓促，文章中难免会出现一些错误或者描述不准确的地方，恳请各位师傅们批评指正。如果你想一起学习 AD 域安全攻防的话，可以加入下面的知识星球一起学习交流。



谢公子

我加入了这个星球，邀你一起学习



域渗透攻防指南

星主：谢公子

60+

成员数量

30+

内容数量

306

运营天数

书籍  《域渗透攻防指南》官方知识星球。
星球讨论与微软AD域相关攻防技术，大家互相交流互相学习，共同进步成长。

现价：¥199

微信扫码加入星球



参考: <https://www.rfc-editor.org/rfc/rfc4120.html>

<https://www.cnblogs.com/artech/archive/2011/01/24/kerberos.html>

[https://docs.microsoft.com/en-us/previous-versions/aa302203\(v=msdn.10\)?redirectedfrom=MSDN#signatures-pac_server_checksum-and-pac_privsvr_checksum](https://docs.microsoft.com/en-us/previous-versions/aa302203(v=msdn.10)?redirectedfrom=MSDN#signatures-pac_server_checksum-and-pac_privsvr_checksum)

<https://docs.microsoft.com/zh-cn/archive/blogs/apgceps/packerberos-2>

https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-pac/166d8064-c863-41e1-9c23-edaaa5f36962

[https://docs.microsoft.com/en-us/previous-versions/aa302203\(v=msdn.10\)#top-level-pac-structure](https://docs.microsoft.com/en-us/previous-versions/aa302203(v=msdn.10)#top-level-pac-structure)

[https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2003/cc772815\(v=ws.10\)?redirectedfrom=MSDN](https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2003/cc772815(v=ws.10)?redirectedfrom=MSDN)

<https://docs.microsoft.com/zh-cn/archive/blogs/openspecification/understanding-microsoft-kerberos-pac-validation>

https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-sfu/3bff5864-8135-400e-bdd9-33b552051d94

相关文章: <https://www.anquanke.com/post/id/171552#h3-5>

<https://zhuanlan.zhihu.com/p/473625225>

<https://www.zhihu.com/question/22177404>